DOCUMENTAÇÃO TÉCNICA

CATI Forms

Plataforma web de criação, compartilhamento e coleta de respostas de formulários.

Resumo da implementação de referência · RF01 a RF16 · CATI Jr.

01 / VISÃO GERAL

O CATI Forms permite que usuários cadastrados criem e consultem formulários, enquanto visitantes enviam respostas por links públicos sem necessidade de conta. As respostas podem ser consultadas e exportadas em CSV e PDF.

Acesso autenticado

Cadastro e login; criação e consulta de todos os formulários e respostas, conforme as regras de acesso do projeto.

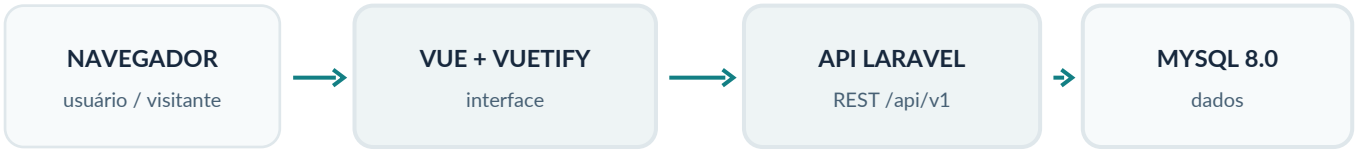
Acesso público

Preenchimento e envio por URL baseada em UUID; sem acesso aos resultados dos demais respondentes.

STACK TECNOLÓGICA

Camada	Tecnologias adotadas
Backend	PHP 8.3+, Laravel 13, Eloquent ORM e Fortify.
Frontend	Vue 3, TypeScript, Inertia.js 3 e Vuetify 3.
Navegação / API	Páginas Inertia + API REST JSON versionada em **/api/v1**.
Banco e infraestrutura	MySQL 8.0 em Docker Compose; Laravel executado no host.
Exportação	CSV via **fputcsv** (PHP); PDF via **dompdf/dompdf**.
Testes / qualidade	Pest, PHPUnit, Pint, PHPStan/Larastan e vue-tsc.

ARQUITETURA RESUMIDA



Inertia entrega páginas; Vue consulta a API; Fortify gerencia sessão; Eloquent persiste dados.

Monólito modular Laravel: as rotas web entregam as páginas via Inertia, e os formulários utilizam **fetch** para consumir endpoints JSON na mesma origem. A autenticação utiliza cookies, sessão e CSRF; não há Bearer token.

02 / FUNCIONALIDADES E DADOS

Os 16 requisitos funcionais foram implementados, agrupados abaixo pelos principais fluxos do produto.

Requisitos	Comportamento entregue
RF01–RF02	**Usuários:** cadastro com username único, autenticação e logout por sessão.
RF03–RF07	**Criação:** título e descrição; perguntas discursivas, múltipla escolha e escolha única com justificativa; configuração de coleta de e-mail.
RF08–RF09	**Compartilhamento:** link público com UUID e preenchimento sem cadastro.
RF10–RF13	**Respostas:** preenchimento obrigatório; justificativa; armazenamento transacional e associações válidas.
RF14	**Consulta:** todos os usuários autenticados visualizam formulários, detalhes e respostas.
RF15–RF16	**Exportação:** download de todas as respostas do formulário em CSV ou PDF.

MODELO DE DADOS

Entidades	Responsabilidade
users	Usuários cadastrados; podem criar formulários.
forms	Título, descrição, UUID público, criador e configuração de e-mail.
questions / question_options	Perguntas de cada formulário e alternativas das questões objetivas.
submissions	Um envio vinculado ao formulário; e-mail opcional.
answers / answer_options	Respostas por pergunta e alternativas efetivamente marcadas.

Relações principais: **formulário → perguntas → alternativas** e **formulário → submissões → respostas → alternativas selecionadas**. As entidades são relacionadas por chaves estrangeiras; a aplicação valida se os identificadores enviados pertencem ao formulário correto.

INTEGRIDADE E REGRAS DE NEGÓCIO

- Formulários são gravados com perguntas e alternativas em **transação**; respostas e marcações também são salvas atomicamente.
- Todas as perguntas são obrigatórias no envio. Escolha única exige **uma alternativa e justificativa**; múltipla escolha aceita várias marcações válidas.
- Formulários não oferecem edição ou exclusão após a criação. Usuários cadastrados têm acesso equivalente aos formulários e resultados.

03 / API REST E SEGURANÇA

A API de formulários usa o prefixo **/api/v1**. Os endpoints administrativos exigem sessão autenticada; os públicos usam o UUID do formulário.

HTTP	Endpoint	Acesso	Finalidade
POST	/forms	Privado	Criar formulário
GET	/forms; /forms/mine	Privado	Listar todos / meus
GET	/forms/{id}	Privado	Detalhes do formulário
GET	/forms/{id}/submissions	Privado	Consultar respostas
GET	/forms/{id}/exports/csv	Privado	Baixar CSV
GET	/forms/{id}/exports/pdf	Privado	Baixar PDF
GET	/public/forms/{uuid}	Público	Acessar formulário
POST	/public/forms/{uuid}/submissions	Público	Enviar respostas

CONTRATO E TRATAMENTO DE ERROS

As respostas JSON usam recursos delimitados; listagens têm paginação de 12 itens. Um envio público válido retorna confirmação sem revelar respostas ou IDs de submissão. Códigos relevantes: **401** (não autenticado), **404** (não encontrado), **419** (CSRF), **422** (validação) e **429** (limite de requisições).

SEGURANÇA IMPLEMENTADA

- **Autenticação:** Fortify, sessão e cookies; senhas armazenadas como hash e username único.
- **Proteção de requisições:** middleware **auth:web** em recursos privados, CSRF em POSTs e validação por Form Requests.
- **Abusos:** limite de 5 tentativas/minuto no login por username/IP e 60 envios públicos/minuto por IP.
- **Integridade:** verificação de perguntas/alternativas e transações nas gravações; consultas parametrizadas via Eloquent.
- **Saída segura:** CSV protege células iniciadas como fórmulas; PDF desativa scripts, PHP e acesso remoto no Dompdf.
- **Dados sensíveis:** respostas e exports não ficam nos endpoints públicos; **.env** e **.env.docker** são arquivos locais ignorados.

EXPORTAÇÃO

CSV: UTF-8, uma linha por submissão, colunas por pergunta e justificativa adicional na escolha única. **PDF:** A4, organização por submissão, perguntas e respostas, com fonte compatível com acentos. Ambos são downloads privados, sem retorno JSON no sucesso.

04 / EXECUÇÃO E VALIDAÇÃO

AMBIENTE LOCAL (RESUMO)

O procedimento completo está no **README.md** do repositório. O Laravel executa no host e acessa o MySQL do Docker pela porta local configurada (exemplo do projeto: **3308**).

```
cp .env.example .env
cp .env.docker.example .env.docker
docker compose --env-file .env.docker up -d db
composer install   # instalar dependências PHP
npm ci --include=optional
php artisan key:generate
php artisan migrate
npm run build && make dev
```

Antes de executar as migrations, alinhe **DB_HOST=127.0.0.1**, **DB_PORT**, nome do banco, usuário e senha do **.env** com o **.env.docker**. O container cria o banco na inicialização de um volume vazio; as migrations criam as tabelas.

TESTES E QUALIDADE

Verificação	Evidência registrada
Suíte automatizada	**271 testes aprovados** e **1.642 asserções** no registro final da implementação.
Isolamento	Pest/Feature usam SQLite **:memory:** com proteções contra execução em banco persistente.
Qualidade	PHPStan, verificação de tipos Vue/TypeScript, Pint/lint e build registrados como aprovados.
Cobertura de cenários	Autenticação, validações, CSRF, persistência, consultas, exportações CSV/PDF e erros.

Resultados extraídos da documentação técnica fornecida. Não houve nova execução da suíte para gerar este PDF.

LIMITAÇÕES E EVOLUÇÃO

- Fora do escopo atual: edição/exclusão de formulários, perfis de acesso, perguntas opcionais e tipos adicionais.
- A API atual usa sessão na mesma origem; integração com sistemas externos e o módulo de Controle de Pessoas ainda não foram implementados.
- Exportação PDF em grande volume depende de memória; não há processamento assíncrono de exportações.
- Implantação de produção requer revisão de HTTPS, backups, configuração de debug e infraestrutura.

REFERÊNCIA

Documento-fonte: **documentacao-tecnica.md** (revisão de 11/11/2026, base **63f896e**). Este PDF é uma síntese; contratos detalhados, exemplos do Postman e passos completos de instalação permanecem na documentação original e no README.